

Roteiro da apresentação

Code Burger's · Qualidade e Teste de Software · Entrega 1 · 12 slides, cerca de 17 minutos

Slides	Quem fala	Parte	Tempo
1	Yuri Mascarenhas	Abertura	1 min
2 e 3	Yuri Mascarenhas	Autenticação, sessões e controle de acesso	3 min
4 e 5	Vinicius Rocha Pinheiro	Carrinho, pedidos e checkout	3 min
6 e 7	Felipe Paixão	Gestão de clientes e endereços	3 min
8 e 9	Vinicius Fonseca de Freitas	Cardápio e montagem de lanches	3 min
10 e 11	Thiago Ferreira	Gestão de estoque e insumos	3 min
12	Thiago Ferreira	Encerramento	1 min

As falas abaixo são as mesmas das notas do apresentador no PowerPoint. Não precisa decorar: use como guia e fale com suas palavras. Quem apresenta deve conseguir abrir o teste citado e explicar o que ele verifica.

Yuri Mascarenhas

Abertura · 1 min

SLIDE 1

Abertura. O sistema é o Code Burger's, uma lanchonete online em Java com Servlets e JDBC, baseado no projeto APS-04. Dividimos em cinco áreas, uma por integrante: cada um apresenta a classe que testou, a estratégia, o resultado e o que encontrou.

Yuri Mascarenhas

Autenticação, sessões e controle de acesso · 3 min

SLIDE 2

Minha parte é autenticação e sessão. A classe é o ValidadorCookie: ela não é CRUD, só recebe os cookies da requisição, procura o cookie certo pelo nome e pergunta ao DaoToken se o token existe. Nos testes o DaoToken é um mock, então nada depende de banco. Além dos casos básicos, testamos as fronteiras: requisição sem cookie, token de cliente tentando passar por funcionário, cookie duplicado e valor malformado. Para demonstrar: abrir ValidadorCookieLimitesTest e rodar `mvn test -Dtest=ValidadorCookie*`.

SLIDE 3

São 26 testes passando e todos os 22 ramos da classe cobertos. O achado mais interessante: procuramos a regra de expiração de token e ela não existe no servidor. A tabela só guarda o texto do token; quem expira é o cookie no navegador, em 30 minutos. Registramos como observação de segurança, não inventamos um resultado. O teste manual é o CT-01, issue 6.

PARA MOSTRAR

Abrir ValidadorCookieLimitesTest e rodar: `mvn test -Dtest=ValidadorCookie*`

Vinicius Rocha Pinheiro

Carrinho, pedidos e checkout · 3 min

SLIDE 4

Minha parte é o carrinho. A classe é o servlet comprar, que tem regra de verdade: valida o cookie, lê o JSON que o carrinho.js monta, separa lanche de bebida, soma o total e grava os vínculos. Mockamos o validador e os quatro DAOs e usamos ArgumentCaptor para olhar o Pedido que chega no DAO. O valor esperado não sai do próprio código: usamos a conta que a tela do carrinho faz, preço vezes quantidade.

SLIDE 5

São 22 testes passando na suíte normal e dois testes que ficam vermelhos de propósito na suíte de defeitos. O primeiro: com 2 lanches de 15 e 1 refrigerante de 5 a tela mostra 35, mas o servlet grava 20, porque soma o preço uma vez por item. O segundo: o id do cliente vem do JSON e nunca é comparado com o cookie. Para demonstrar: `mvn test -Pdefeitos -Dtest=ComprarRegrasTest`.

PARA MOSTRAR

Rodar: `mvn test -Pdefeitos -Dtest=ComprarRegrasTest` e mostrar 'expected: <35.0> but was: <20.0>'

Felipe Paixão

Gestão de clientes e endereços · 3 min

SLIDE 6

Minha parte é o cadastro de clientes. O DaoCliente é um DAO, mas o login tem uma decisão: só entra se o hash MD5 bater e o cliente estiver ativo. E o salvar reaproveita um endereço que já existe. Completamos com duas classes que não são CRUD: o servlet cadastro e o EncryptadorMD5. Simplificamos o teste antigo, que usava Objenesis e reflexão, para injeção por construtor. O hash esperado vem dos vetores da RFC, inclusive o MD5 de 'a', que começa com zero e testa o preenchimento.

SLIDE 7

São 29 testes passando. Defeitos: o servidor grava cliente com nome, usuário e senha vazios, e a validação da tela também não funciona por causa de uma condição sempre falsa no JavaScript. Outro: a busca por usuário concatena texto no SQL, então um apóstrofo quebra o login. O MD5 foi tratado como comportamento legado, só caracterizamos. O CT-03 manual ainda está pendente, issue 8.

PARA MOSTRAR

Abrir DaoClienteTest (deveRecusarClienteInativo) e mostrar em cadastro.js a condição `!field.name == 'complemento'`

Vinicius Fonseca de Freitas

Cardápio e montagem de lanches · 3 min

SLIDE 8

Minha parte é a montagem de lanches. O DaoLanche é CRUD, então a regra que escolhemos está no servlet salvarLancheCliente: ele calcula o preço somando valor de venda vezes quantidade de cada ingrediente, grava o lanche como inativo e deveria vincular os ingredientes. O teste calcula o preço esperado à mão: 2 queijos, 3 bacons e 1 pão dá 18,50.

SLIDE 9

Tínhamos uma suspeita de que os ingredientes não eram salvos. Investigamos: o DaoLanche faz o INSERT normalmente. O problema é no servlet, que usa o mesmo Iterator em dois laços; o primeiro consome tudo e o segundo nunca executa. O teste espera 3 vínculos e recebe zero. No teste manual CT-04, feito em 20 de setembro, o botão Adicionar não respondeu, e isso virou a issue 17. Demonstração: `mvn test -Pdefeitos -Dtest=SalvarLancheClienteTest`.

PARA MOSTRAR

Mostrar os dois laços `while(keys.hasNext())` em salvarLancheCliente.java e rodar: `mvn test -Pdefeitos -Dtest=SalvarLancheClienteTest`

Thiago Ferreira

Gestão de estoque e insumos · 3 min

SLIDE 10

Minha parte é insumos e estoque. Aqui somos transparentes: o DaoIngrediente é CRUD e o sistema não tem regra de estoque, a compra nem dá baixa. Então cobrimos o DAO, que era a issue 5, e escolhemos como classe não CRUD os servlets de insumo, que verificam se quem chama é funcionário e convertem o JSON. Mostramos que um token de cliente não entra no painel.

SLIDE 11

São 16 testes passando e os 6 ramos do DAO cobertos. O defeito: quantidade e valores negativos são aceitos e gravados, sem validação no servlet nem restrição no banco. Não afirmamos que o preço de venda precisa ser maior que o de compra, porque nada no sistema sustenta essa regra. O CT-05 manual ainda está pendente, issue 10.

PARA MOSTRAR

Abrir IngredienteControllersTest (deveBloquearQuemNaoEhFuncionario e caracterizacaoValoresNegativosSaoAceitos)

Thiago Ferreira

Encerramento · 1 min

SLIDE 12

Fechamento. A suíte tem 115 testes unitários passando. Separamos 5 testes que descrevem o comportamento correto e ficam vermelhos enquanto os defeitos existirem; eles rodam com mvn test -Pdefeitos. No total documentamos 6 defeitos. Sendo honestos sobre os limites: dois testes manuais ainda não foram executados, a área de estoque não tem uma classe de regra de negócio porque o sistema não tem essa regra, e não corrigimos nenhum defeito, porque o objetivo da entrega era testar e reportar.

PARA MOSTRAR

Rodar mvn clean test e mostrar 'Tests run: 115, Failures: 0'

Perguntas prováveis

Qualquer integrante pode responder

POR QUE ALGUNS TESTES FICAM VERMELHOS DE PROPÓSITO?

Eles descrevem o comportamento correto. Enquanto o defeito existir, falham. Ficam no perfil -Pdefeitos para não quebrar o build normal, e não usamos @Disabled para não esconder o problema.

VOCÊS CORRIGIRAM OS DEFEITOS?

Não. O objetivo da entrega era testar e reportar; cada defeito tem um teste que o reproduz.

POR QUE MEXERAM NO CÓDIGO DE PRODUÇÃO?

Só adicionamos construtores que recebem a conexão ou os DAOs, para poder usar mocks. O construtor padrão e o comportamento no Tomcat não mudaram.

A CLASSE DA ÁREA DE ESTOQUE NÃO É CRUD?

DaoIngrediente é CRUD, e assumimos isso. O sistema não tem regra de estoque; usamos os servlets de insumo como melhor alternativa e registramos a lacuna.

O QUE A COBERTURA GARANTE?

Só que aquelas linhas e ramos foram executados pelos testes, não que não existam defeitos.